

UNIDAD 3



DISEÑO Y MODELADO EN POO

UML, PATRONES Y ARQUITECTURA MVC



Objetivo

Comprender cómo modelar y diseñar sistemas orientados a objetos utilizando **UML**, aplicar **patrones de diseño** y organizar aplicaciones con el **patrón MVC**.



Modela tus ideas



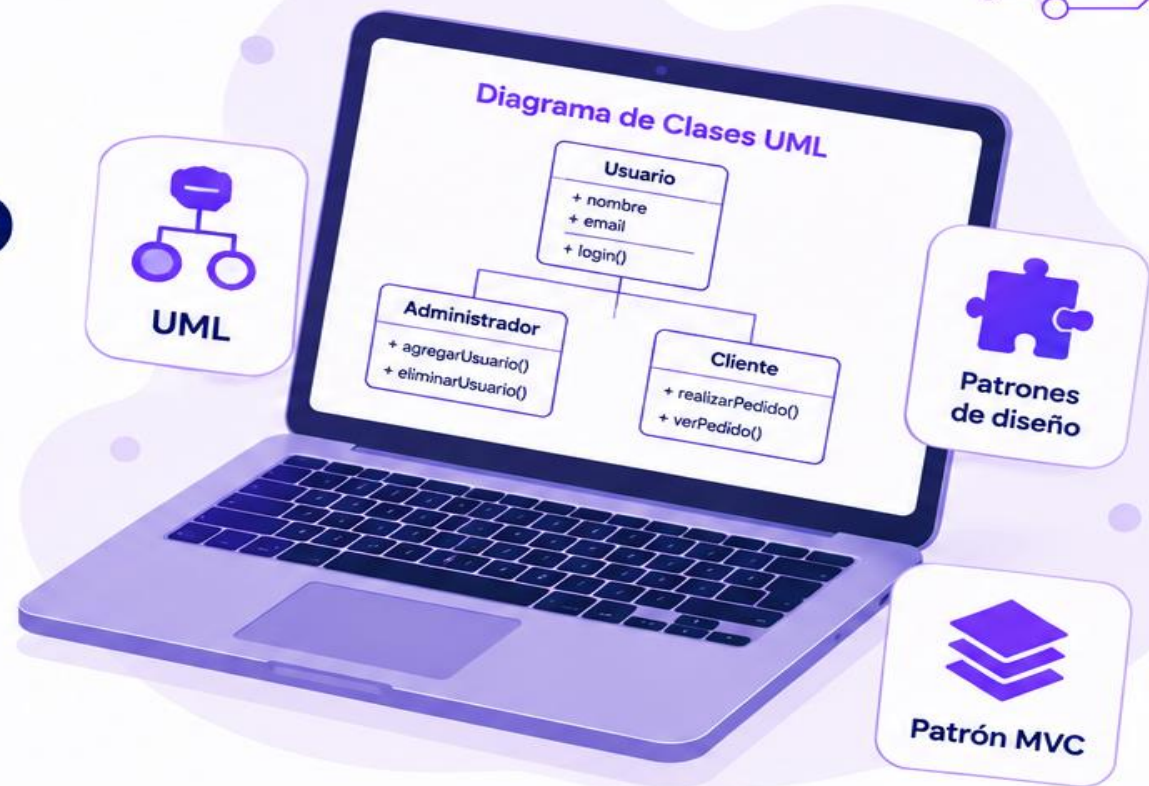
Aplica patrones comprobados



Organiza tu aplicación



Desarrolla mejor y más rápido



Temas de la unidad

3.1 Introducción al UML

Representar y visualizar sistemas usando diagramas.

3.2 Diseño y Patrones en POO

Usar soluciones reutilizables para problemas comunes.

3.3 Introducción al Patrón MVC

Separar responsabilidades para aplicaciones más organizadas y escalables.

3.1

Introducción al UML



¿Qué es UML?

UML (Unified Modeling Language) es un lenguaje gráfico que permite representar y planificar sistemas antes de programarlos.



¿Para qué sirve?

Ayuda a visualizar, diseñar y comunicar la estructura de un sistema de manera sencilla y entendible.



Ejemplo de la vida real

Antes de construir una casa se realiza un plano.



Antes de crear un software se realiza un diagrama UML.



¿Por qué usar UML?



Visualiza el sistema



Mejora la comunicación



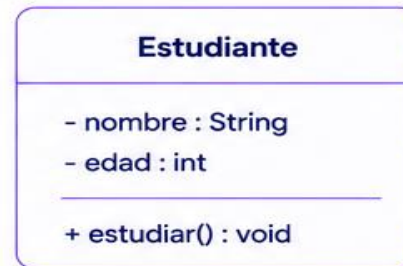
Reduce errores antes de programar



Ahorra tiempo y esfuerzo



Ejemplo UML sencillo (Diagrama de clase)



Nombre de la clase
Estudiante

Atributos
nombre, edad

Método
estudiar()



Ejemplo en Java

```

class Estudiante {
    String nombre;
    int edad;

    void estudiar() {
        System.out.println("Estudiando");
    }
}
  
```

Atributos
nombre, edad

Método
estudiar()



Idea clave

UML ayuda a **visualizar** cómo será el programa antes de escribir código.



3.2

Diseño y Patrones en POO



¿Qué es un patrón?

Un patrón es una solución reutilizable a un problema común que ocurre una y otra vez en la programación.



¿Para qué sirve?

Para evitar "reinventar la rueda" y usar soluciones ya probadas que funcionan bien.



Ejemplo de la vida real

Cuando construimos una casa, seguimos modelos ya conocidos.



En programación, sucede lo mismo.



Ejemplo

Sistema de biblioteca



Sin patrón

Crear todo desde cero cada vez.



Con patrón

Utilizar una estructura organizada y ya probada.



Beneficios



Código más organizado

Todo tiene su lugar.



Menos errores

Las soluciones ya fueron probadas.



Más fácil mantenimiento

Entender y modificar el código es más sencillo.



Reutilización

Puedes usar la misma solución en distintos proyectos.



Ejemplos de patrones comunes



Singleton

Una sola instancia de una clase.



Factory

Crea objetos sin especificar la clase exacta.



Observer

Notifica cambios a múltiples objetos.



Strategy

Cambia el comportamiento sin cambiar la clase.



Idea clave

Los patrones son **guías** que ayudan a desarrollar software de manera ordenada, eficiente y profesional.



3.3

Introducción al Patrón MVC



MVC

¿Qué es MVC?

MVC (Model – View – Controller) es un patrón que divide una aplicación en tres partes que trabajan juntas.



¿Para qué sirve?

Para organizar el código, separar responsabilidades y facilitar el mantenimiento del sistema.

1 Modelo (Model)

Gestiona los datos y la lógica del negocio.

Ejemplo

```
class Estudiante {
    private String nombre;
    private int edad;

    public Estudiante(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
    // Getters y Setters...
}
```



Se encarga de los datos.
No muestra información en pantalla.

2 Vista (View)

Muestra la información al usuario.

Ejemplo



Información del Estudiante

Nombre: Ana
Edad: 20 años



Se encarga de la interfaz.
Solo muestra datos.

3 Controlador (Controller)

Recibe las solicitudes del usuario, procesa los datos y decide qué mostrar.

Ejemplo

```
class EstudianteController {
    private Estudiante modelo;

    public EstudianteController(Estudiante modelo) {
        this.modelo = modelo;
    }

    public void mostrarEstudiante() {
        // Lógica para obtener datos del modelo
        // y enviarlos a la vista
    }
}
```



Recibe la acción del usuario.
Comunica el Modelo con la Vista.

Flujo de trabajo



Idea clave

- ✓ Modelo: maneja los datos.
- ✓ Vista: muestra la información.
- ✓ Controlador: conecta y coordina.

MVC = separación que hace más fácil entender, mantener y escalar aplicaciones.



3.4

Ejemplo Completo Integrando UML y MVC

Veamos cómo UML y el patrón MVC trabajan juntos con un ejemplo sencillo: **Sistema de Estudiantes**

1 Paso 1: UML

Diseñamos la clase Estudiante con UML.

Estudiante

```
- nombre : String
- edad : int

+ estudiar() : void
```



UML nos ayuda a planificar cómo será la estructura del sistema.

2 Paso 2: Modelo

Creamos la clase en código (Java).

```
class Estudiante {
    private String nombre;
    private int edad;

    public Estudiante(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
    // Getters y Setters...
}
```



El Modelo guarda y maneja los datos del estudiante.

3 Paso 3: Vista

Mostramos la información al usuario.



Información del Estudiante

Nombre: Ana Pérez
Edad: 20 años



La Vista presenta los datos de forma clara y amigable.

4 Paso 4: Controlador

El Controlador recibe la acción del usuario y decide qué hacer.

```
class EstudianteController {
    private Estudiante modelo;

    public EstudianteController(
        Estudiante modelo) {
        this.modelo = modelo;
    }

    public void mostrarEstudiante() {
        // Obtiene datos del modelo
        // y los envía a la vista
    }
}
```



El Controlador conecta el Modelo con la Vista y gestiona las acciones del usuario.

5 Resultado Final

Así interactúan las partes trabajando juntas.



Usuario



Vista



Controlador



Modelo



¿Qué sucede?



El usuario realiza una acción (ej: presiona un botón)



El controlador procesa la acción



El modelo obtiene o actualiza los datos



La vista muestra los resultados al usuario



Resumen final

- ✓ UML nos permite planificar el sistema visualmente.
- ✓ Patrones de diseño nos dan soluciones probadas y reutilizables.
- ✓ MVC organiza el código en 3 partes que trabajan juntas.
- ✓ Resultado: aplicaciones más organizadas, fáciles de mantener y escalar.



3.5

Resumen: UML + Patrones + MVC en acción

Un ejemplo sencillo para entender cómo todo se conecta.

1 Paso 1: UML

Diseñamos la clase Estudiante en un diagrama.

Diagrama de clase

Estudiante

```
- nombre : String
- edad : int

+ estudiar() : void
```



UML es el plano del sistema.

2 Paso 2: Modelo

Creamos la clase en código (Modelo).

```
class Estudiante {
    private String nombre;
    private int edad;

    public Estudiante(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```



El Modelo guarda y gestiona los datos.

3 Paso 3: Vista

Mostramos la información al usuario.

Vista (Interfaz)



Información del Estudiante

Nombre: Ana Pérez
Edad: 20 años



La Vista presenta los datos de forma clara y amigable.

4 Paso 4: Controlador

El Controlador recibe la acción del usuario y coordina.

```
class EstudianteControlador {
    private Estudiante modelo;

    public EstudianteControlador(Estudiante modelo) {
        this.modelo = modelo;
    }

    public void mostrarEstudiante() {
        // Obtiene datos del modelo
        // y los envía a la vista
    }
}
```



El Controlador conecta el Modelo con la Vista.

5 Paso 5: Resultado

Así interactúan las partes trabajando juntas.

Flujo de interacción



Usuario
Realiza una acción (ej. hacer clic)



Vista
Muestra la información



Controlador
Procesa la acción



Modelo
Gestiona los datos



¿Qué aprendimos?

- ✓ UML nos ayuda a planificar y visualizar el sistema.
- ✓ Los patrones de diseño nos dan soluciones probadas y reutilizables.
- ✓ MVC organiza el código en 3 partes que trabajan juntas.
- ✓ Esto hace que las aplicaciones sean más fáciles de desarrollar, entender, mantener y escalar.



Resumen final



Planificar

UML diseña el sistema.



Diseñar

Patrones nos dan buenas prácticas.



Organizar

MVC estructura el código.



Resultado

Aplicaciones mejores, más rápidas y escalables.



Idea clave

Con UML, Patrones y MVC trabajamos de forma ordenada, eficiente y profesional para crear software de calidad.





GRACIAS

— POR TU ATENCIÓN —

...

